

PUC Minas

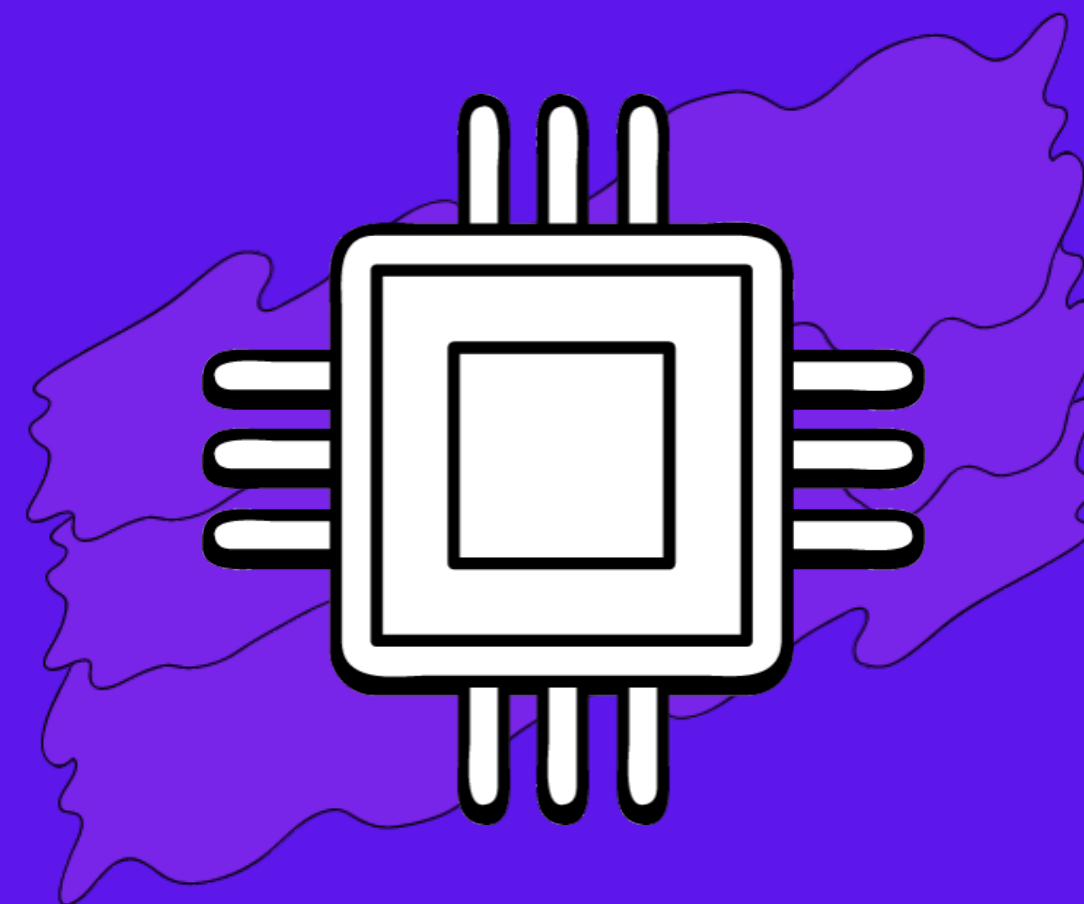
Introdução

JAVA E POO



Conteúdo

- 1 Introdução a Java
- 2 Orientação a Objetos
- 3 Exercícios
- 4 Dicas Trabalho Prático



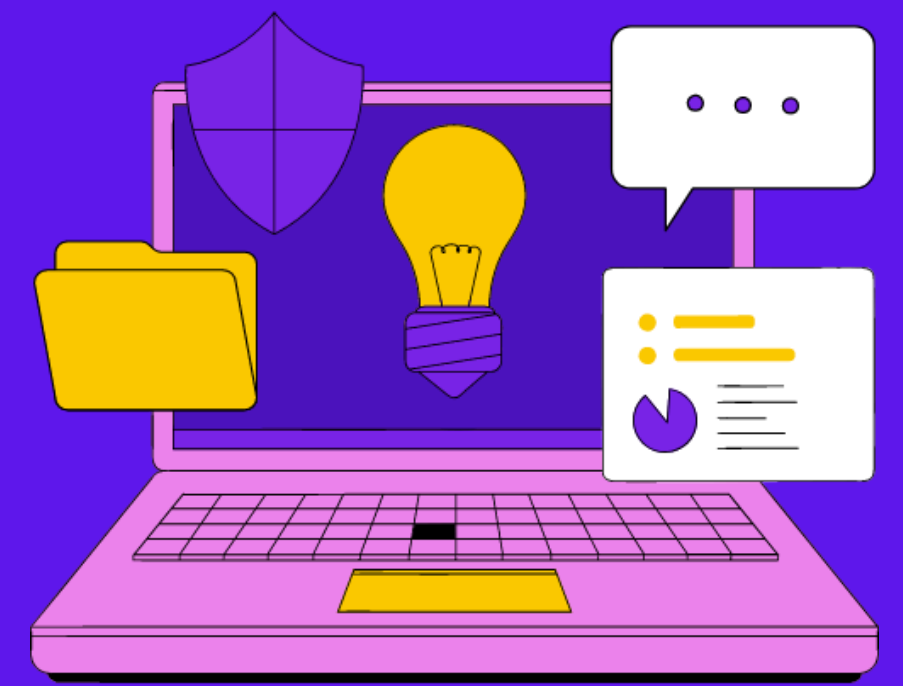
Sintaxe Java

Saída de Dados

Saída de dados

Diferença entre o printf, print e println.

```
1  public class Main {  
    Run | Debug | Tabnine | Edit | Test | Fix | Explain | Document  
2      public static void main(String[] args) {  
3          System.out.printf();  
4          System.out.print();  
5          System.out.println();  
6      }  
7  }
```



Saída de dados

```
1 public class Main {  
    Run | Debug | Tabnine | Edit | Test | Explain | Document  
2     public static void main(String[] args) {  
3         int n = 10;  
4         System.out.printf(format:"O número digitado é %d \n", n);  
5         System.out.print("O número digitado é " + n + "\n");  
6         System.out.println("O número digitado é " + n);  
7  
8         System.out.print(s:"Fim");  
9     }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS



java Main

```
O número digitado é 10  
O número digitado é 10  
O número digitado é 10  
Fim
```

No printf, a sintaxe é igual a C.

No print, para printar a variável necessita de usar o operador '+' para concatenar a variável na frase. Detalhe: o **print** não quebra a linha ao final da frase.

No println, ele também utiliza o operador '+' para concatenar a variável na frase. Detalhe: o **println** quebra a linha ao final.

Entrada de datos

Entrada de dados

Vale ressaltar que Java é um linguagem orientada a objetos.

Então para alguns casos, para um operação que em C era simples, em Java é necessário criar um objeto do tipo específico.

Por exemplo, em vários casos é necessário usar **new** para criar os objetos. (assim como em C++ que era possível usar **new**).

Entrada de dados

Para ler entrada de dados em Java, primeiramente precisaremos de importar a seguinte biblioteca: **java.util.Scanner**;

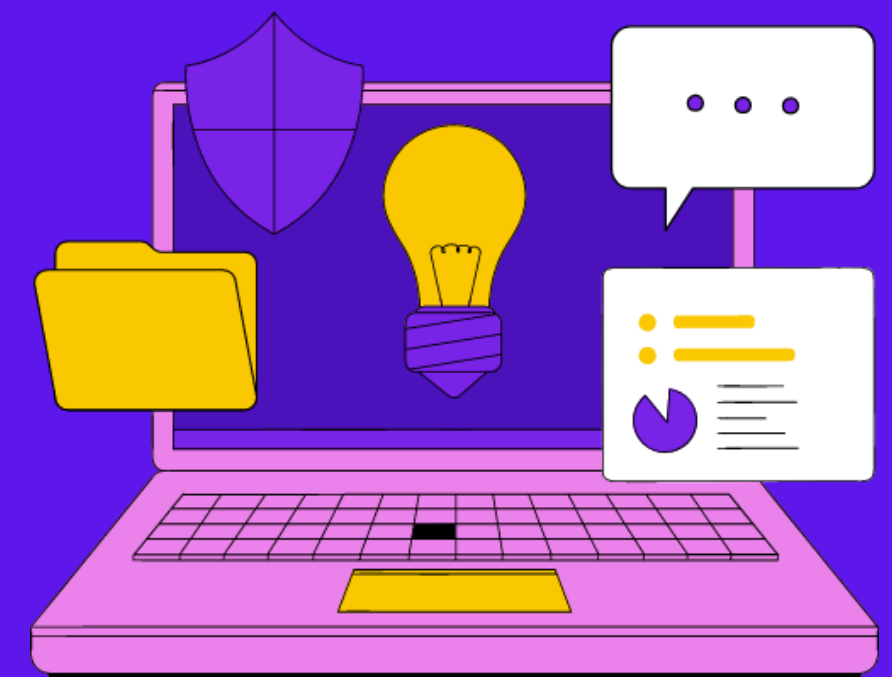
E dentro da sua main ou função, caso desejar, você precisa chamar o scanner com:

Scanner [nome da variável] = new Scanner(System.in);

E ao final do código, ou quando não precisar mais de utilizá-lo em uma fu, é necessário fechá-lo com: **[nome da variável].close();**

Entrada de datos

```
1  import java.util.Scanner;
2
3  public class Main {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          sc.close();
8      }
9  }
```



Entrada de datos

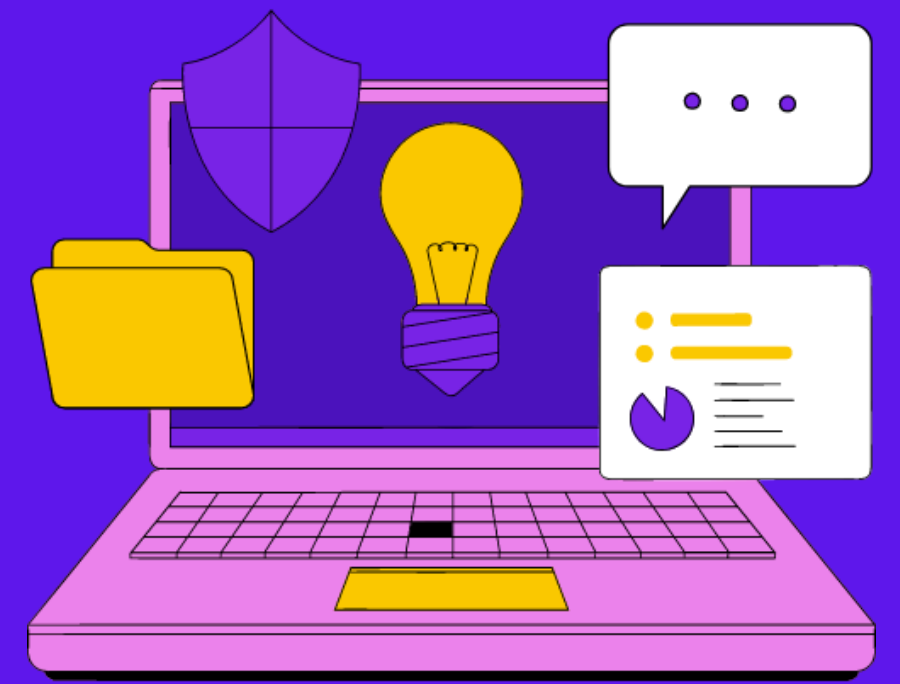
int ?

float ?

double ?

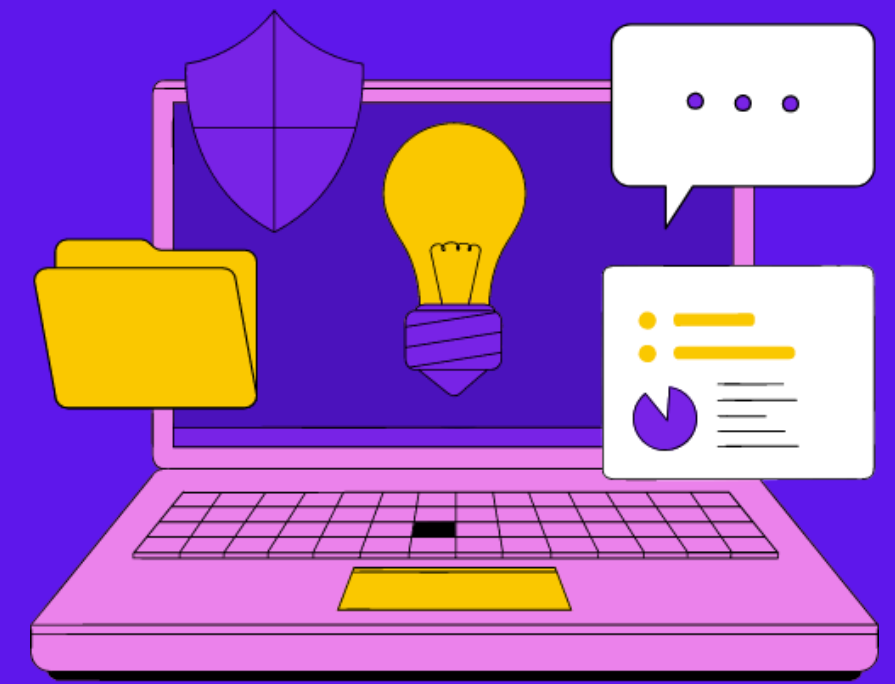
string ?

char ?



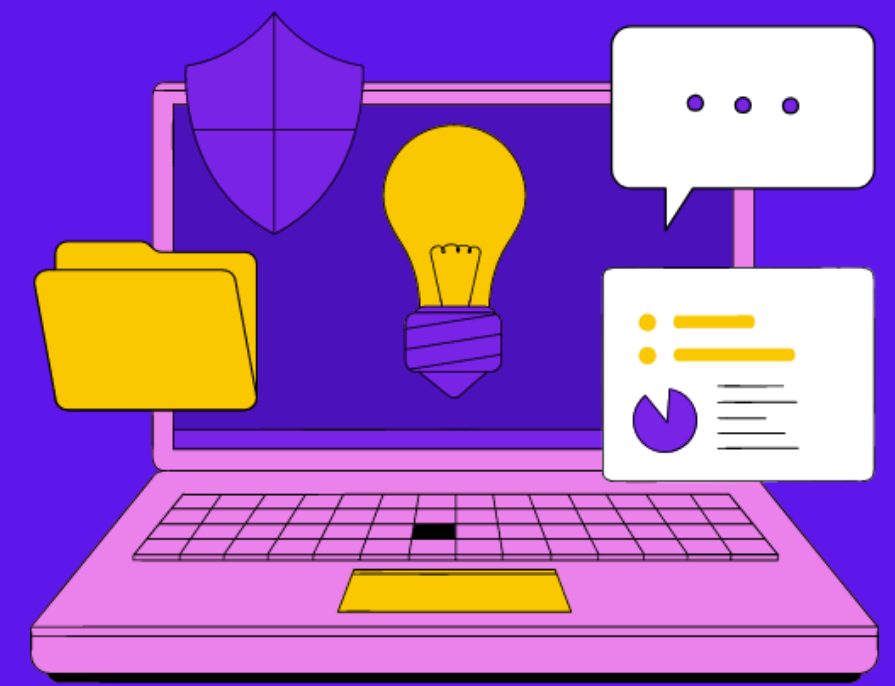
Entrada de datos - int

```
1  import java.util.Scanner;
2
3  public class Main {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          int n = sc.nextInt();
9
10         sc.close();
11     }
```



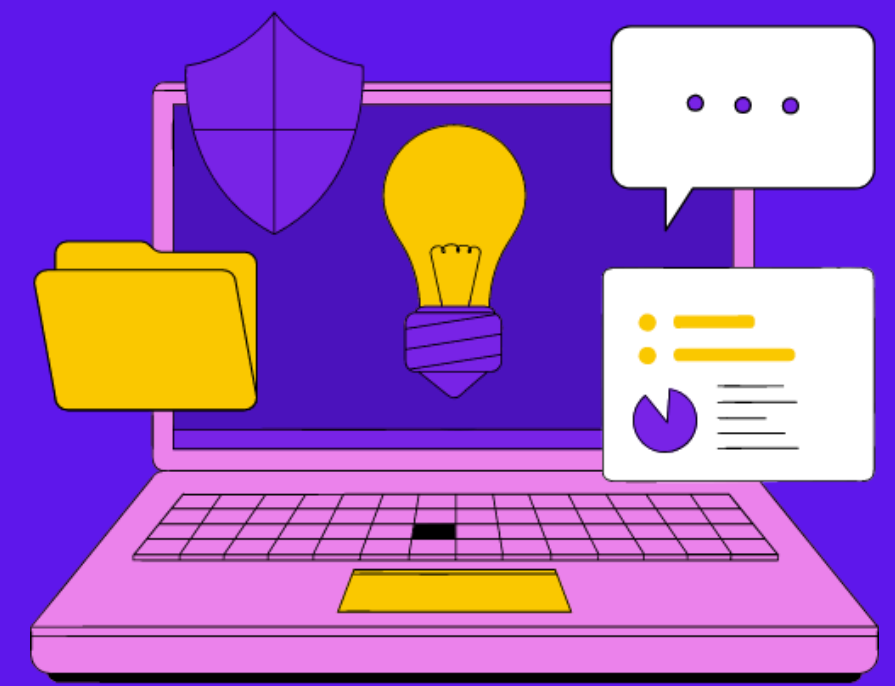
Entrada de datos - float

```
1  import java.util.Scanner;
2
3  public class Main {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          float n = sc.nextFloat();
9
10         sc.close();
11     }
```



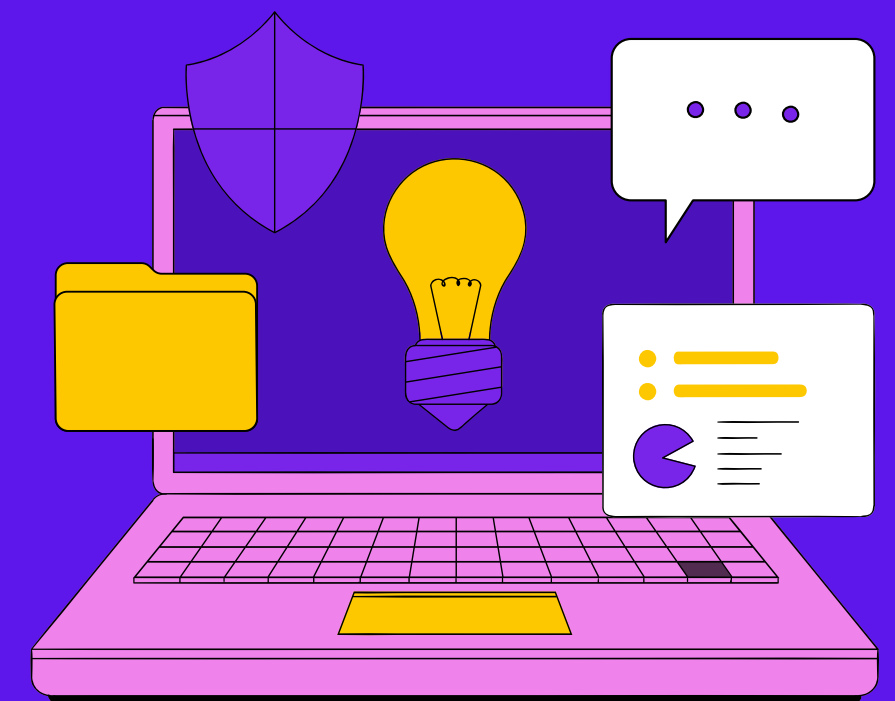
Entrada de datos - double

```
1  import java.util.Scanner;
2
3  public class Main {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          double n = sc.nextDouble();
9
10         sc.close();
11     }
```



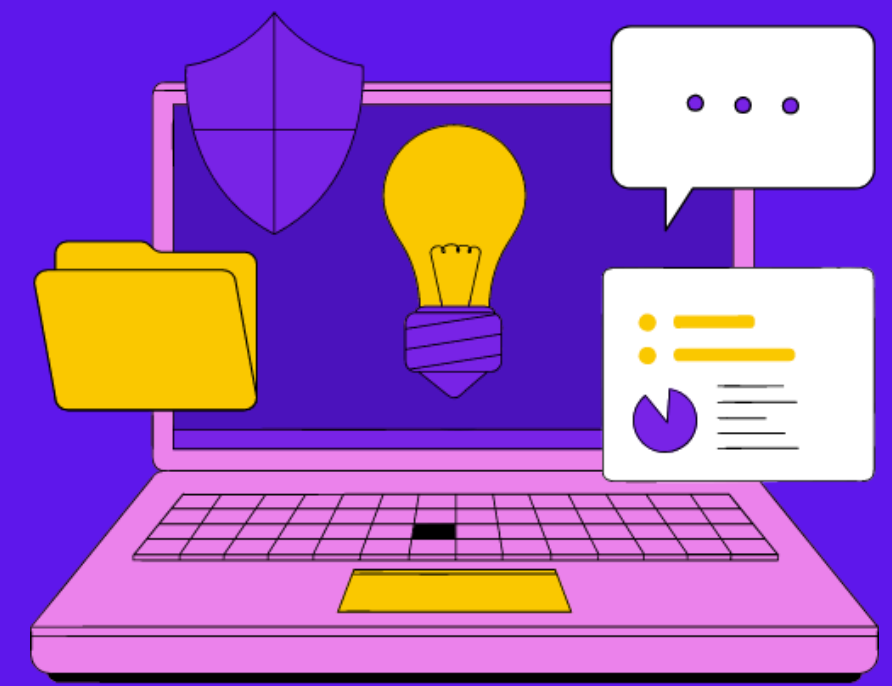
Entrada de dados - string

```
1  import java.util.Scanner;
2
3  public class Main {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          // Lê apenas uma palavra
9          String str = sc.next();
10
11          sc.close();
12      }
13  }
```



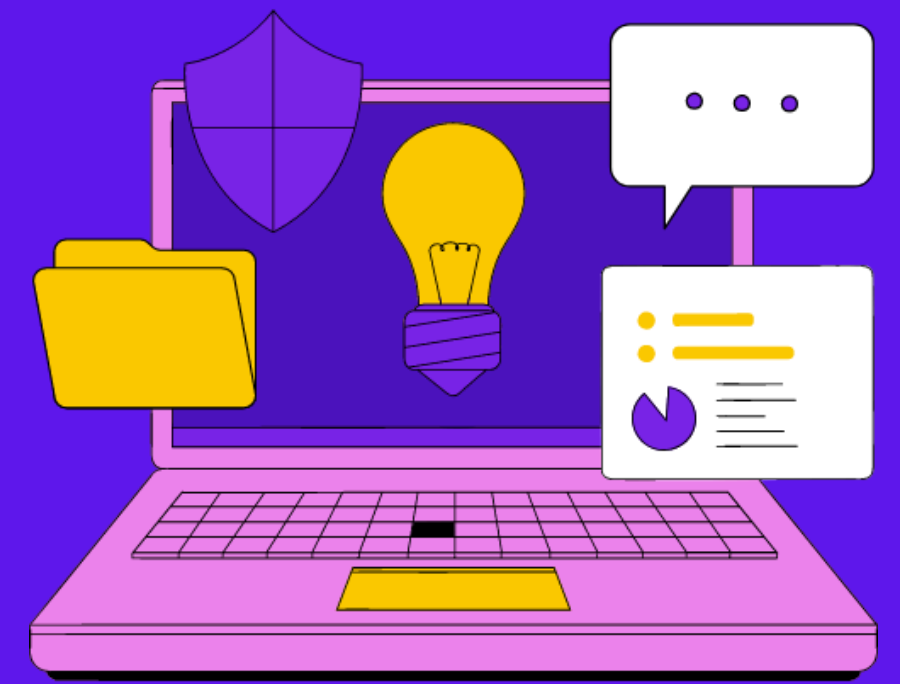
Entrada de dados - string

```
1  import java.util.Scanner;
2
3  public class Main {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          // Lê a frase até o \n (quebra de linha)
9          String str = sc.next();
10
11          sc.close();
12      }
13  }
```



Entrada de datos - char

```
1  import java.util.Scanner;
2
3  public class Main {
4      Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          char letra = sc.next().charAt(0);
8
9          sc.close();
10     }
11 }
```



Declaração de array

Declaração de array

Para iniciar um array, é necessário criar um objeto do tipo array.

[tipo] [nome variável] = new [tipo][tamanho array];

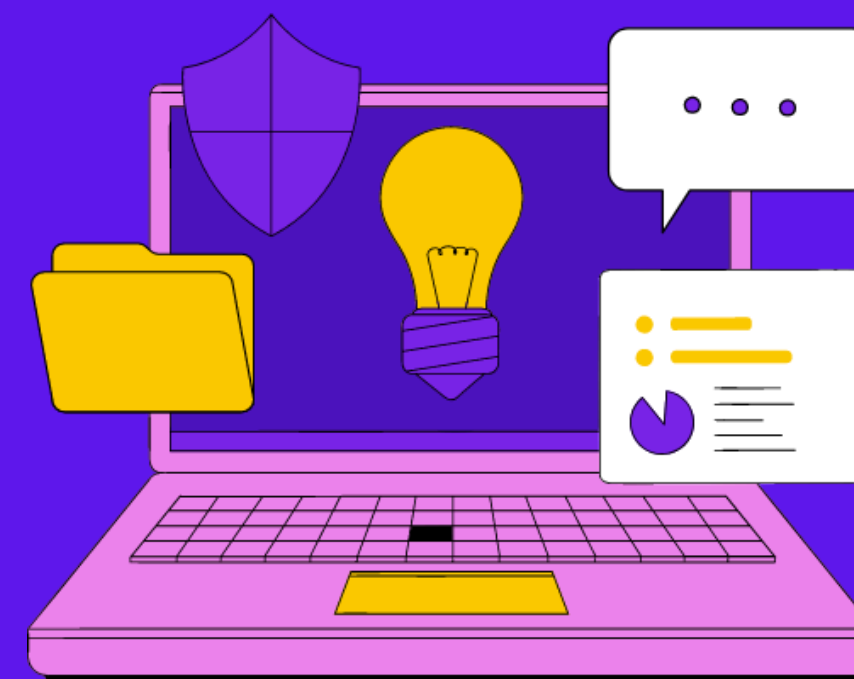
int[] ?

float[] ?

double[] ?

char[] ?

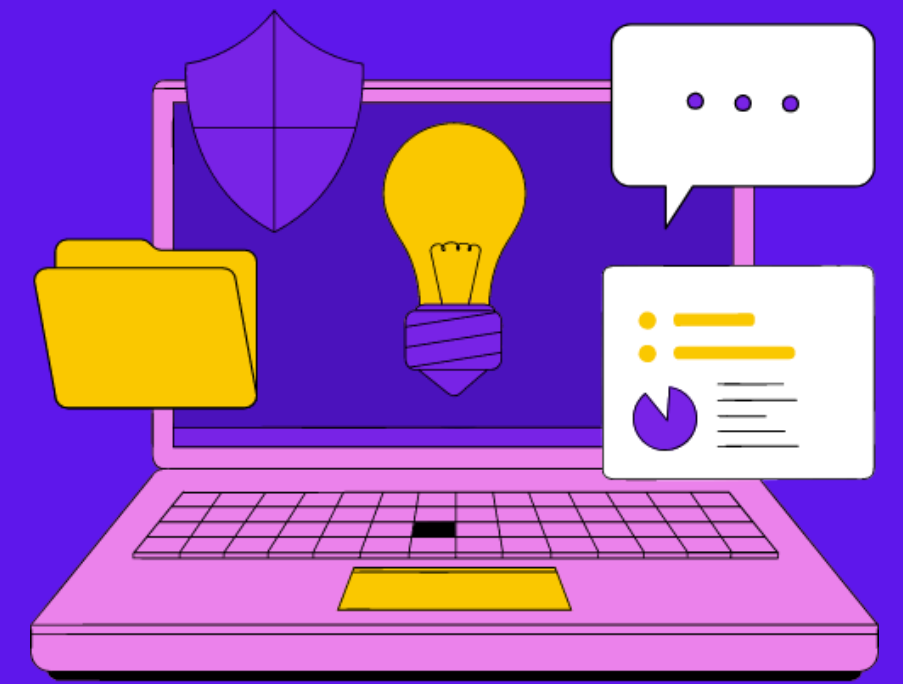
string[] ?



Declaração de array - int

Três maneiras de fazer, ambas funcionam, basta escolher a maneira necessária para cada caso.

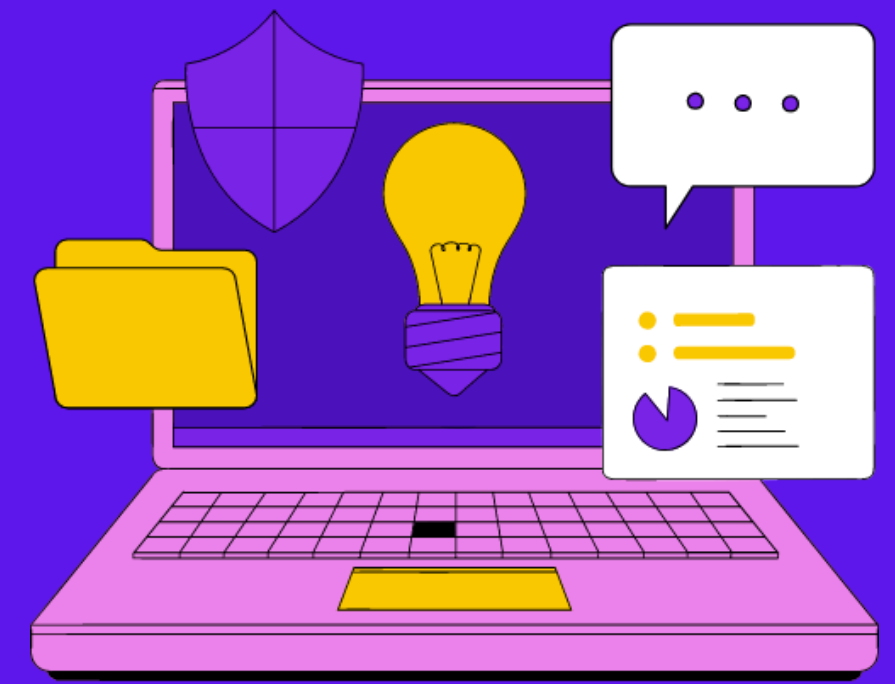
```
1  public class Main {  
    Run | Debug | Tabnine | Edit | Test | Fix | Explain | Document  
2      public static void main(String[] args) {  
3          int n = 10;  
4          int[] array1 = new int[n]; // tamanho 10 vazio  
5          int[] array2 = new int[10]; // tamanho 10 vazio  
6          int[] array3 = new int[]{0, 1, 2, 3, 4, 5, 6, 7, 8, 9};  
7          // tamanho 10 com valores inicializados  
8      }  
9  }
```



Declaração de array - float

Três maneiras de fazer, ambas funcionam, basta escolher a maneira necessária para cada caso.

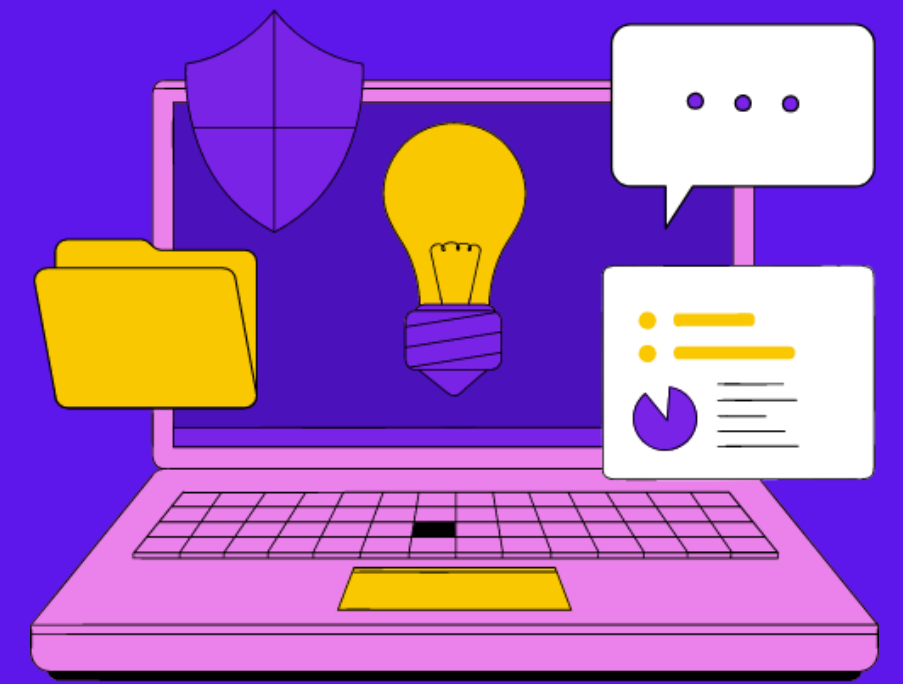
```
1  public class Main {  
    Run | Debug | Tabnine | Edit | Test | Fix | Explain | Document  
2  public static void main(String[] args) {  
3      int n = 10;  
4      float[] array1 = new float[n]; // tamanho 10 vazio  
5      float[] array2 = new float[10]; // tamanho 10 vazio  
6      float[] array3 = new float[]{0, 1, 2, 3, 4, 5, 6, 7, 8, 9};  
7      // tamanho 10 com valores inicializados  
8  }  
9  }
```



Declaração de array - double

Três maneiras de fazer, ambas funcionam, basta escolher a maneira necessária para cada caso.

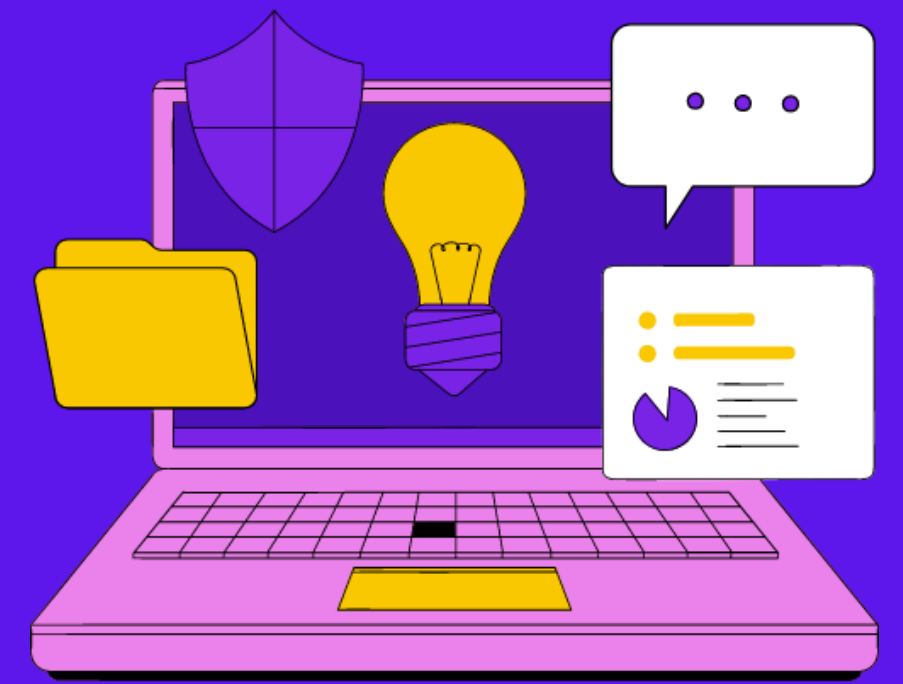
```
1  public class Main {  
    Run | Debug | Tabnine | Edit | Test | Fix | Explain | Document  
2      public static void main(String[] args){  
3          int n = 10;  
4          double[] array1 = new double[n]; // tamanho 10 vazio  
5          double[] array2 = new double[10]; // tamanho 10 vazio  
6          double[] array3 = new double[]{0.0, 1.1, 2.2, 3.3, 4.4, 5.5};  
7          // tamanho 6 valores inicializados  
8      }  
9  }
```



Declaração de array - char

Três maneiras de fazer, ambas funcionam, basta escolher a maneira necessária para cada caso.

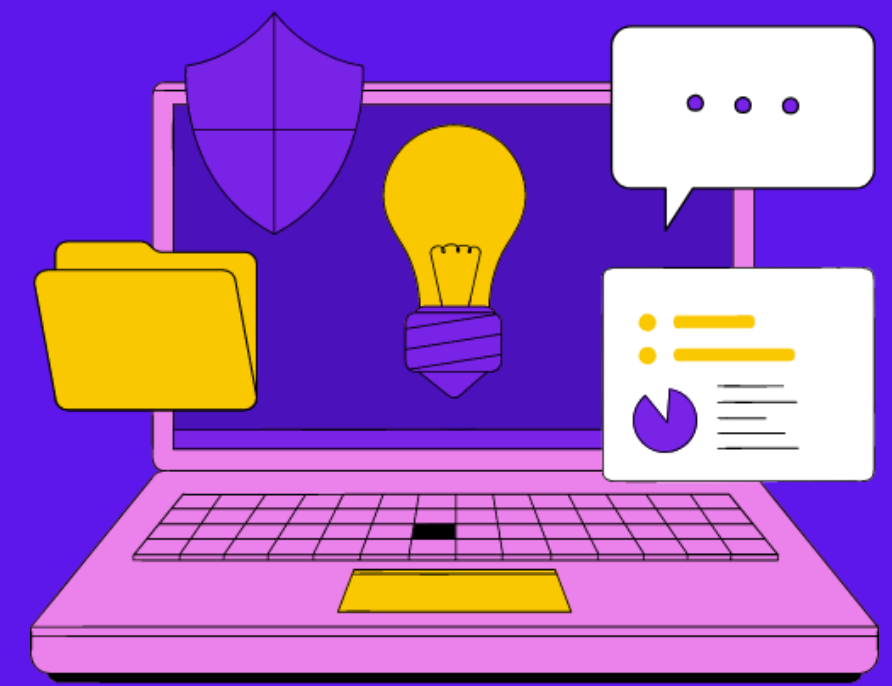
```
1  public class Main {  
    Run | Debug | Tabnine | Edit | Test | Fix | Explain | Document  
2  public static void main(String[] args){  
3      int n = 10;  
4      char[] array1 = new char[n]; // tamanho 10 vazio  
5      char[] array2 = new char[10]; // tamanho 10 vazio  
6      char[] array3 = new char[]{'a', 'b', 'c', 'd', 'e', 'f'};  
7      // tamanho 6 valores inicializados  
8  }  
9  }
```



Declaração de array - string

Três maneiras de fazer, ambas funcionam, basta escolher a maneira necessária para cada caso.

```
1 public class Main {  
    Run | Debug | Tabnine | Edit | Test | Fix | Explain | Document  
2     public static void main(String[] args){  
3         int n = 10;  
4         String[] array1 = new String[n]; // tamanho 10 vazio  
5         String[] array2 = new String[10]; // tamanho 10 vazio  
6         String[] array3 = new String[]{"Zero", "Um", "Dois", "Três", "Quatro", "Cinco"};  
7         // tamanho 6 valores inicializados  
8     }  
9 }
```



Leitura e Escrita de Arquivos

Leitura e Escrita de Arquivos

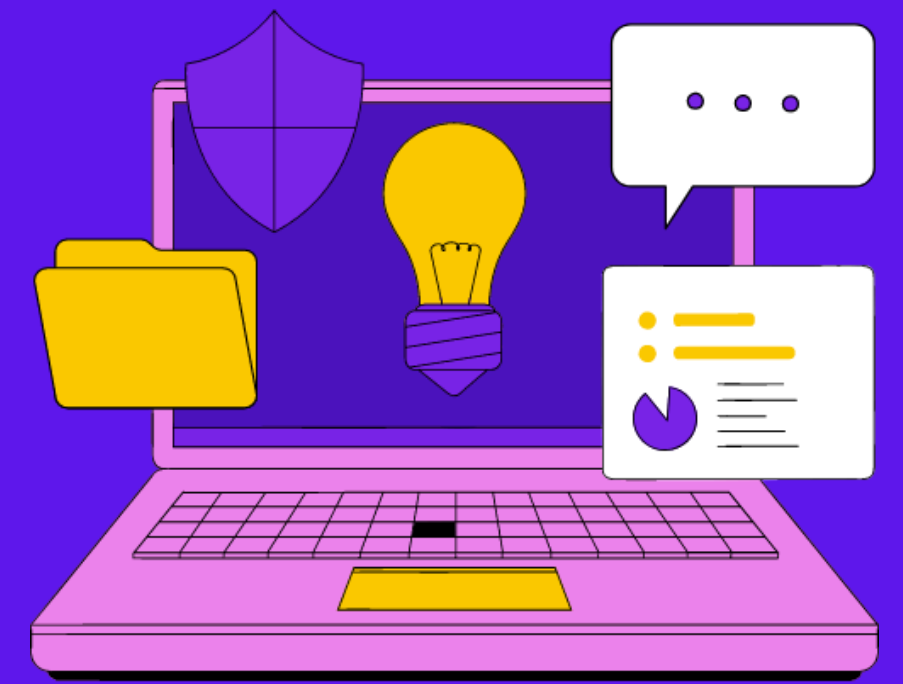
Para leitura e escrita de arquivos existem algumas maneiras como **Buffered**, **File**, **Print**, etc. mas irei citar somente duas mais comuns. Tais como:

FileReader (leitura)

BufferedReader (leitura)

FileWriter (escrita)

BufferedWriter (escrita)

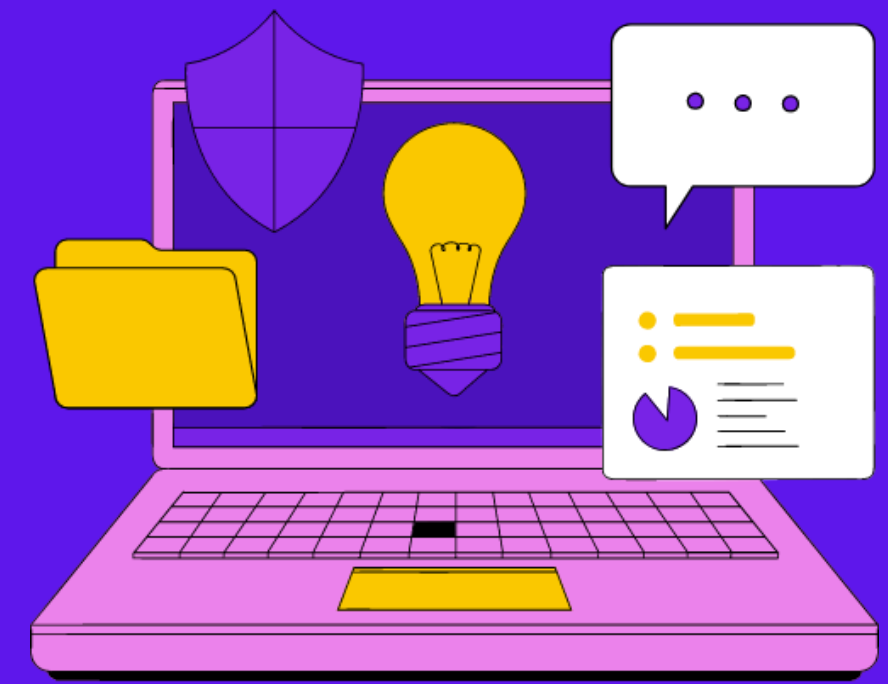


Leitura de Arquivos

Leitura de Arquivos

Leitura de arquivos utilizando **FileReader**.

```
1  import java.io.FileReader;
2  import java.io.IOException;
3
4  public class Main {
5      Run | Debug | Tabnine | Edit | Test | Fix | Explain | Document
6      public static void main(String[] args) throws IOException {
7          try {
8              FileReader fr = new FileReader(fileName:"dados.txt");
9
10             int c; // cada caractere lido vem como int (código Unicode, tabela ASCII)
11             StringBuilder sb = new StringBuilder();
12
13             while ((c = fr.read()) != -1) { // -1 = fim do arquivo
14                 sb.append((char) c); // converte/força inteiro -> char e acumula/concatena
15             }
16
17             fr.close();
18
19             // Conversão do conteúdo salvo na variável sb do tipo StringBuilder para uma String
20             String conteudo = sb.toString();
21             System.out.println("Conteúdo lido:\n" + conteudo);
22         } catch (IOException e) {
23             System.out.println(e.getMessage());
24         }
25     }
26 }
```



Leitura de Arquivos

Leitura de arquivos utilizando **BufferedReader**.

```
1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class Main {
6      Run | Debug | Tabnine | Edit | Test | Explain | Document
7      public static void main(String[] args) throws IOException {
8          try {
9              // O BufferedReader necessita de um FileReader ou outro Reader para funcionar
10             BufferedReader br = new BufferedReader(new FileReader(fileName:"dados.txt"));
11
12             String linha;
13             while ((linha = br.readLine()) != null) { // Lê linha por linha até o final do arquivo
14                 System.out.println(linha);
15             }
16
17             br.close();
18         } catch (IOException e) {
19             e.printStackTrace();
20         }
21     }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

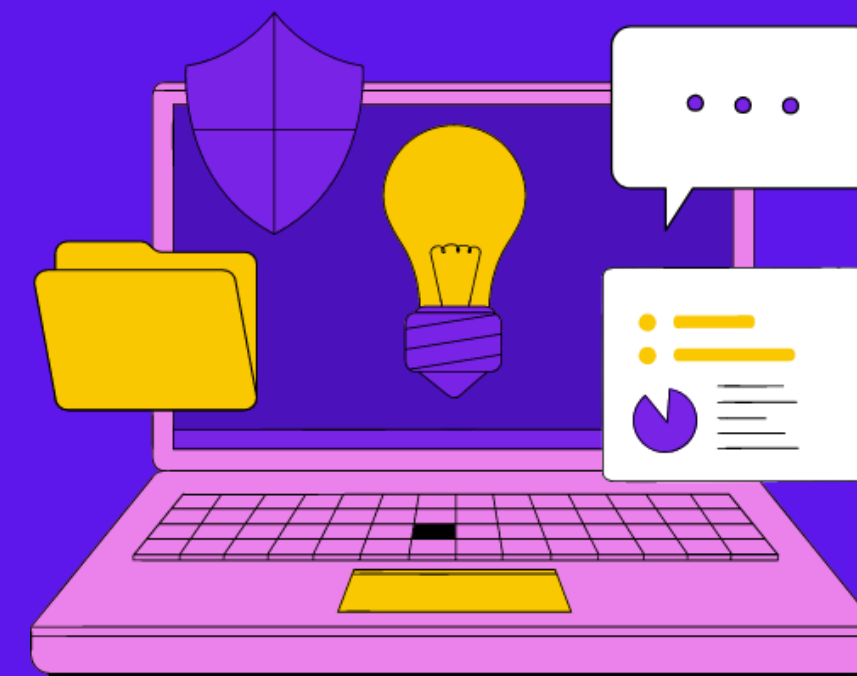
 powershell + v 

java Main

Linha 1

Linha 2

Linha 3

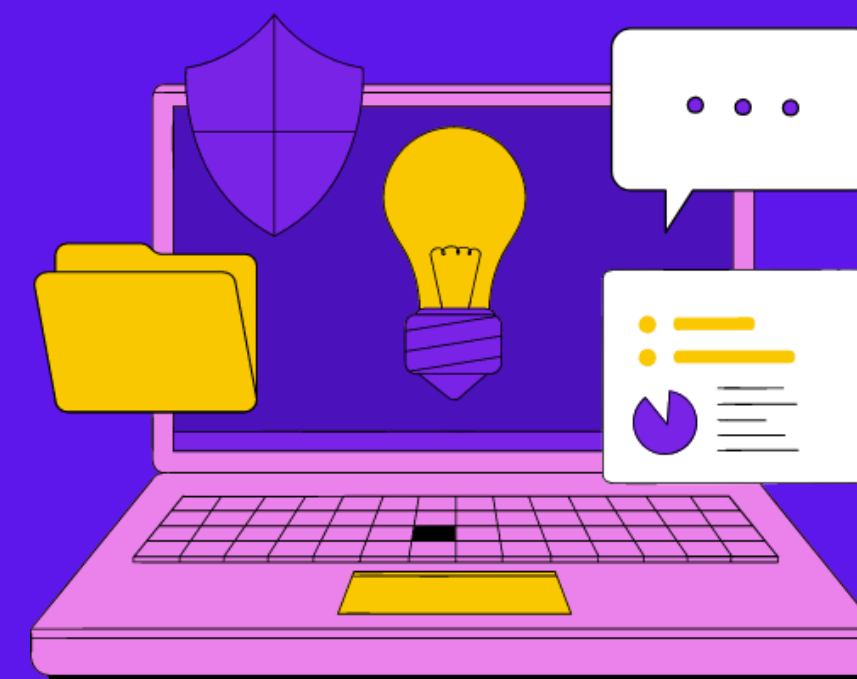


Escrita de Arquivos

Escrita de Arquivos

Escrita de arquivos utilizando **FileWriter**.

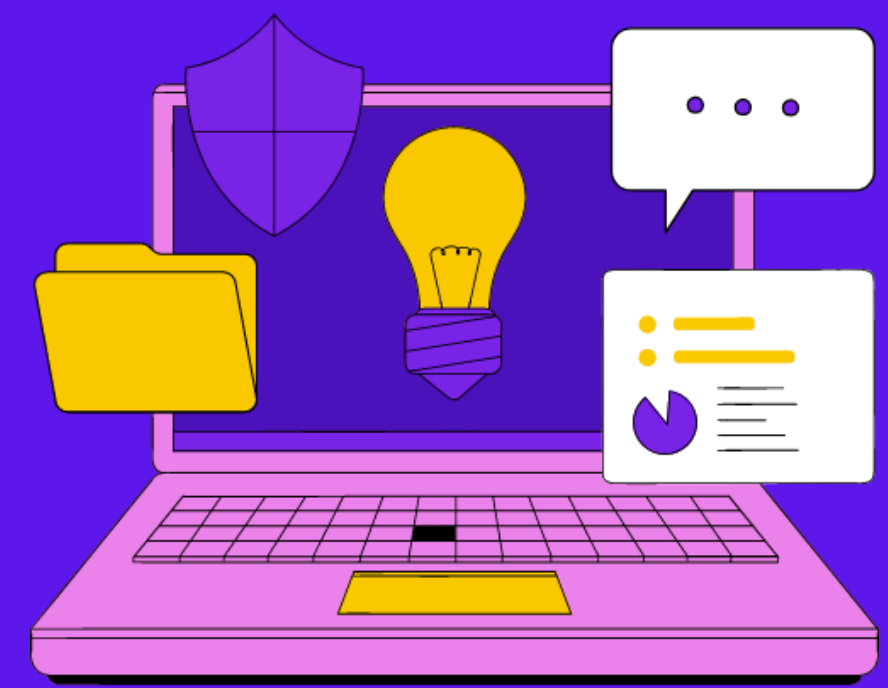
```
1  import java.io.FileWriter;
2  import java.io.IOException;
3
4  public class Main {
5      Run | Debug | Tabnine | Edit | Test | Explain | Document
6      public static void main(String[] args) throws IOException {
7          try {
8              FileWriter fw = new FileWriter(fileName:"dados.txt"); // cria/abre arquivo
9
10             fw.write(str:"Linha 1\n");
11             fw.write(str:"Linha 2\n");
12             fw.write(str:"Linha 3\n");
13
14             fw.close();
15         } catch (IOException e) {
16             e.printStackTrace();
17         }
18     }
```



Escrita de Arquivos

Escrita de arquivos utilizando **BufferedWriter**.

```
1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4
5  public class Main {
    Tabnine | Edit | Test | Explain | Document | Run | Debug
6      public static void main(String[] args) throws IOException {
7          try {
8              // O BufferedWriter necessita de um FileWriter ou outro Writer para funcionar
9              BufferedWriter bw = new BufferedWriter(new FileWriter(fileName:"saida.txt"));
10
11              bw.write(str:"Linha 1"); // Escreve no arquivo
12              bw.newLine(); // Adiciona quebra de linha
13              bw.write(str:"Linha 2");
14              bw.newLine();
15              bw.write(str:"Linha 3");
16              bw.newLine();
17
18              bw.close();
19          } catch (IOException e) {
20              e.printStackTrace();
21          }
22      }
```

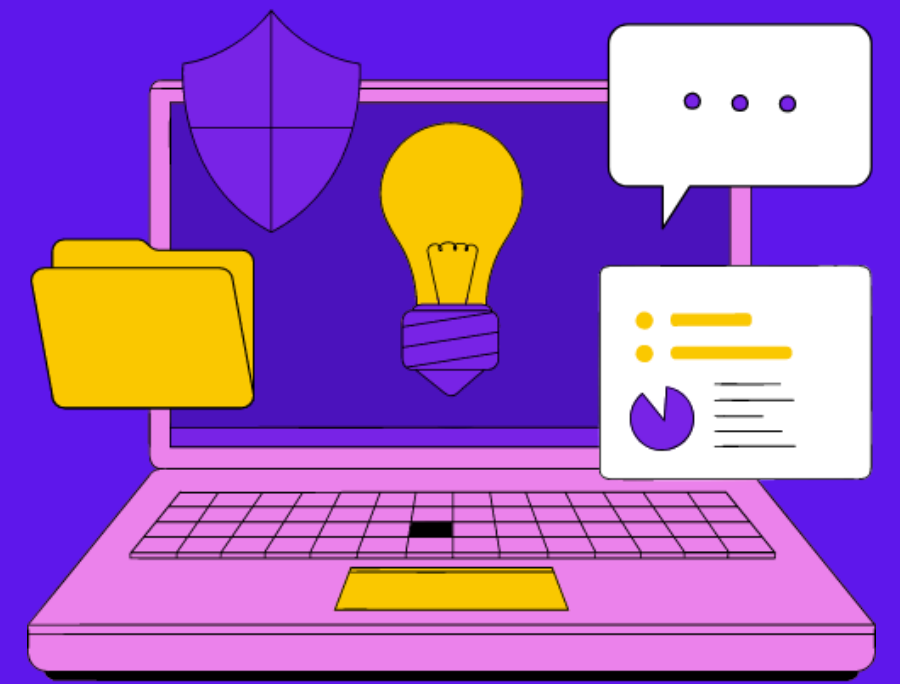


Orientação a Objetos

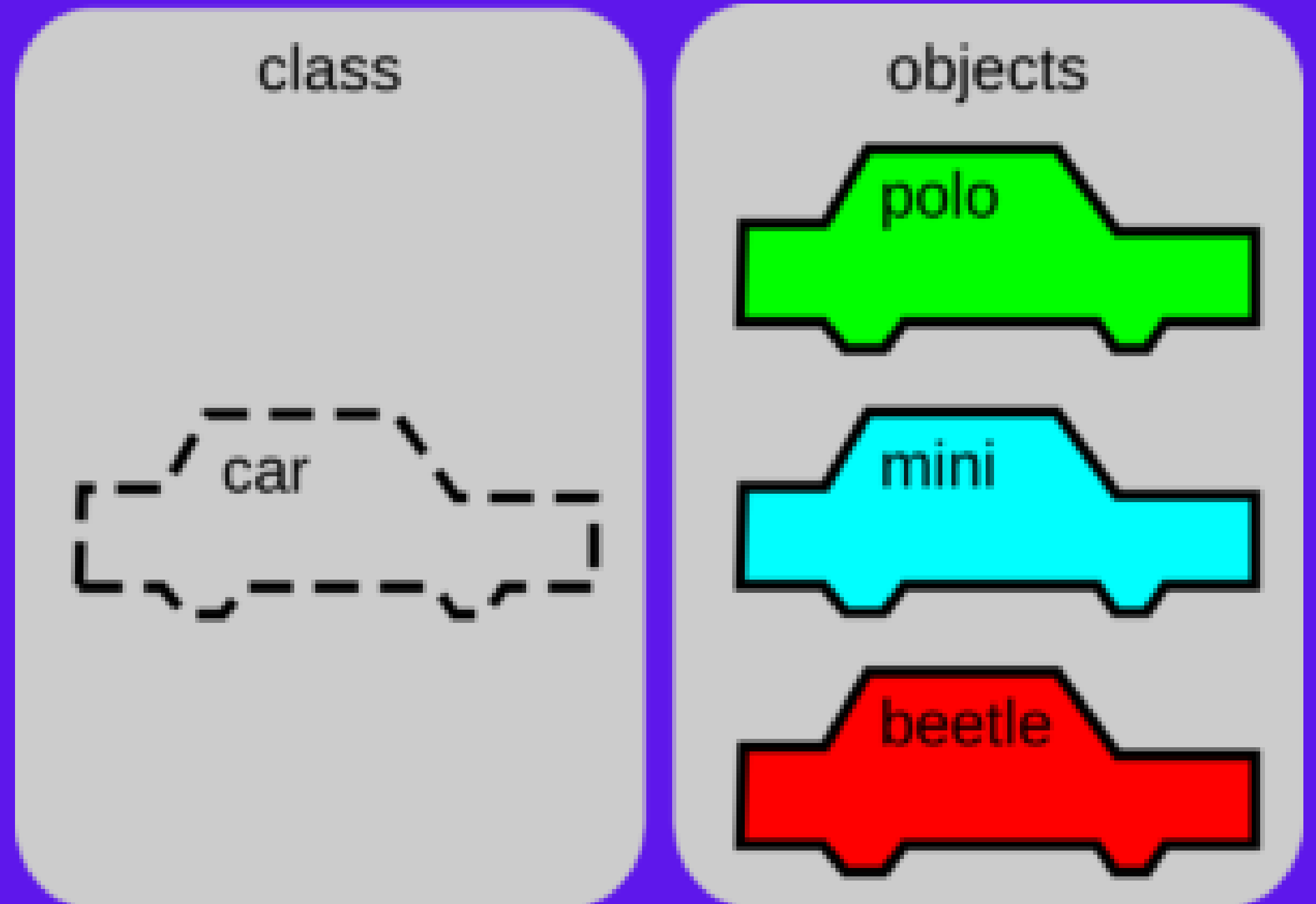
Abstração

Abstrair um conceito seria o equivalente a reduzir um objeto/conceito/entidade a suas características mais essenciais.

Na orientação a objetos, nós vamos usar da abstração de conceitos através de classes e objetos para criar uma experiência mais rica de programação.



Observe como um conceito abstraído de carro, consegue ser particularizado em diferentes modelos de carro.



Aprendendo na Prática

Observe que eu tento reduzir um objeto complexo a algumas características simples e que dentro dessas classes eu tenho atributos, ações e um construtor.

```
class Pessoa{
    private String sexo;
    private int idade;

    public Pessoa(String sexo, int idade){
        this.sexo = sexo;
        this.idade = idade;
    }

    public static void leLivro(Livro livro){
        System.out.println("Pessoa está lendo" + livro.getTitulo());
    }
}
```

```
class Livro {
    private String titulo;
    private String genero;
    private double preco;
}
```

```
public class Biblioteca{
    private String nome;
    private String endereco;
}
```

Aprendendo na Prática

Vamos pensar no caso de uma pessoa, agora com essa classe eu sou capaz de criar outras pessoas seguindo esse mesmo “template”, as pessoas criadas a partir desses templates são as instâncias. Essas instâncias vão herdar as ações e atributos, no entanto esses atributos e ações podem ser personalizados, por exemplo: cada instância de pessoa pode ter um sexo e idade diferente.

```
class Pessoa{
    private String sexo;
    private int idade;

    public Pessoa(String sexo, int idade){
        this.sexo = sexo;
        this.idade = idade;
    }
}
```

```
public class Biblioteca{
    private String nome;
    private String endereco;

    Run | Debug
    public static void main(String[] args) {
        Pessoa bibliotecario = new Pessoa(sexo:"masculino", idade:44);
        Pessoa leitor = new Pessoa(sexo:"feminino", idade:16);
    }
}
```


Aprendendo na Prática

Toda classe precisa de um construtor, o construtor é responsável por criar o novo objeto segundo as características passadas. Uma classe pode ter mais de um construtor.

```
class Pessoa{  
    private String sexo;  
    private int idade;  
  
    public Pessoa(String sexo, int idade){  
        this.sexo = sexo;  
        this.idade = idade;  
    }  
}
```

```
public Pessoa(String sexo, int idade){  
    this.sexo = sexo;  
    this.idade = idade;  
}
```

Encapsulamento

```
class Livro {  
    private String titulo;  
    private String genero;  
    private double preco;  
  
    public String getTitulo() {  
        return titulo;  
    }  
    public void setTitulo(String titulo) {  
        this.titulo = titulo;  
    }  
    public String getGenero() {  
        return genero;  
    }  
    public void setGenero(String genero) {  
        this.genero = genero;  
    }  
}
```

Encapsular é ocultar a implementação interna do mundo exterior. Isso aumenta a segurança e a capacidade de dar manutenção no código. Um exemplo são os GETS e SETS.

Herança

Uma classe filho herda os métodos e atributos de sua classe pai. Podemos entender a classe filho como um tipo, uma especificação da classe pai.

```
class Pessoa{  
    private String sexo;  
    private int idade;  
  
    public Pessoa(String sexo, int idade){  
        this.sexo = sexo;  
        this.idade = idade;  
    }  
}
```

```
class Bibliotecario extends Pessoa{  
    private double salario;  
    public Bibliotecario(String sexo, int idade, double salario){  
        super(sexo, idade);  
        this.salario = salario;  
    }  
}
```

```
class Seguranca extends Pessoa{  
    private String turno;  
  
    public Seguranca(String sexo, int idade, String turno){  
        super(sexo, idade);  
        this.turno = turno;  
    }  
}
```


Herança

```
class Bibliotecario extends Pessoa{
    private double salario;
    public Bibliotecario(String sexo, int idade, double salario){
        super(sexo, idade);
        this.salario = salario;
    }

    public void pedeSilencio(){
        System.out.println("SHHHHhhhhh!!!!");
    }
}
```

Observe que é possível adicionar atributos e métodos independentes dos da classe pai, e que para acessar itens da classe pai no construtor usamos o super.

Polimorfismo

1) Se métodos tiverem o mesmo nome, mas tiverem parâmetros diferentes eles são considerados métodos diferentes.

```
public class Prints {  
    public static void printa(int numero){  
        System.out.println(numero);  
    }  
  
    public static void printa(String texto){  
        System.out.println(texto);  
    }  
}
```